Light

Boolean Logic & Truth Tables

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Logic gates
2. Truth tables
3. Logic circuits & expressions
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Computer Systems: Boolean Logic**. This topic covers logic gates (NOT, AND, OR, XOR), truth tables, logic circuits, and Boolean expressions. This content is assessed in **Paper 2 (Component 2): Computing concepts**.
Exam questions typically ask you to **construct** truth tables for logic gates and circuits, **create and interpret** logic circuit diagrams, and **write or interpret** Boolean expressions. Expect questions worth 2—6 marks requiring both understanding and practical application.

Why Boolean logic matters

Boolean logic is the foundation of all digital computing. Every decision a computer makes, from simple calculations to complex AI, ultimately comes down to combinations of true/false (1/0) operations. Understanding Boolean logic helps you:

- **Understand how processors work:** CPUs are built from billions of logic gates performing Boolean operations
- **Write better programs:** Conditional statements (if/else) use Boolean logic to make decisions
- **Design digital circuits:** Electronic systems use logic gates to process signals

- **Solve logical problems:** Breaking complex conditions into simple true/false components
  The AQA specification requires you to work with four logic gates (NOT, AND, OR, XOR), construct truth tables with up to three inputs, and convert between logic circuits and Boolean expressions.

What you must know

This checklist covers the key concepts for Boolean logic. You should be able to perform each skill confidently.

- **Construct truth tables** for NOT, AND, OR, and XOR logic gates.
- **Construct truth tables** for simple logic circuits using combinations of gates (up to three inputs).
- **Interpret** the results of simple truth tables.
- **Create, modify and interpret** simple logic circuit diagrams using NOT, AND, OR, and XOR gates.
- **Recognise** the standard symbols for NOT, AND, OR, and XOR gates.
- **Create and interpret** simple Boolean expressions using NOT, AND, OR, and XOR operations.
- **Convert** a Boolean expression into a logic circuit.
- **Convert** a logic circuit into a Boolean expression.

> **Specification note.** AQA states that students do not need to know about or use NAND and NOR logic gates. You only need to work with NOT, AND, OR, and XOR. Truth tables may have up to three inputs.

Toolkit

The four logic gates

NOT
Inverts the input

AND
Both inputs must be 1

OR
At least one input is 1

XOR
Exactly one input is 1

Boolean expression operators

AQA uses these symbols in Boolean expressions:

Gate	Symbol	Example	Meaning
AND	**.** (dot)	A . B	A AND B
OR	**+** (plus)	A + B	A OR B
XOR	⊕ (circled plus)	A ⊕ B	A XOR B
NOT	**overbar** (line above)	$\bar{A}$ or $\overline{A}$	NOT A

Example: (A AND B) OR (NOT C) is written as: $\mathbf{A \cdot B + \overline{C}}$

Truth table quick reference

NOT

A	Output
0	**1**
1	**0**

AND

A	B	Output
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

OR

A	B	Output
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

XOR

A	B	Output
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

Exam tip: Remember the key differences: AND requires BOTH inputs to be 1. OR requires AT LEAST ONE input to be 1. XOR requires EXACTLY ONE input to be 1 (not both). NOT simply inverts: 0 becomes 1, 1 becomes 0.

1. Logic gates

A **logic gate** is an electronic component that takes one or more binary inputs and produces a single binary output based on a specific rule. Logic gates are the building blocks of all digital circuits, from simple calculators to complex processors.

1.1 NOT gate

The **NOT gate** (also called an inverter) has one input and one output. It inverts the input: if the input is 0, the output is 1; if the input is 1, the output is 0.
Rule: Output = opposite of input
Symbol: Triangle with a small circle (bubble) at the output

```
          ┌─────┐
A ────────┤     ├o──────── Q
          │  1  │
          └─────┘

Input A │ Output Q
────────┼─────────
   0    │    1
   1    │    0
```

The small circle represents "NOT" (inversion). The triangle with "1" inside is the standard symbol.

1.2 AND gate

The **AND gate** has two or more inputs and one output. The output is 1 only when ALL inputs are 1. If any input is 0, the output is 0.
Rule: Output = 1 only if A AND B are both 1
Symbol: D-shape (flat on the left, curved on the right)

```
          ┌─────┐
A ────────┤     │
          │  &  ├──────── Q
B ────────┤     │
          └─────┘

A │ B │ Output Q
──┼───┼─────────
0 │ 0 │    0
0 │ 1 │    0
1 │ 0 │    0
1 │ 1 │    1
```

Think of AND as multiplication: 0 × anything = 0. Only 1 × 1 = 1.

1.3 OR gate

The **OR gate** has two or more inputs and one output. The output is 1 when AT LEAST ONE input is 1. The output is 0 only when all inputs are 0.
Rule: Output = 1 if A OR B (or both) are 1
Symbol: Curved shield shape (pointed at output)

```
          ╭──────╮
A ────────┤      │
          │ ≥1   ├──────── Q
B ────────┤      │
          ╰──────╯

A │ B │ Output Q
──┼───┼──────────
0 │ 0 │    0
0 │ 1 │    1
1 │ 0 │    1
1 │ 1 │    1
```

Think of OR as "at least one" — any 1 in the inputs gives a 1 output.

1.4 XOR gate (Exclusive OR)

The **XOR gate** (Exclusive OR) has two inputs and one output. The output is 1 when EXACTLY ONE input is 1. If both inputs are the same (both 0 or both 1), the output is 0.
Rule: Output = 1 if inputs are different
Symbol: Like OR but with an extra curved line at the input

```
          ╭╭──────╮
A ────────┤│      │
          │| =1   ├──────── Q
B ────────┤│      │
          ╰╰──────╯

A │ B │ Output Q
──┼───┼──────────
0 │ 0 │    0
0 │ 1 │    1
1 │ 0 │    1
1 │ 1 │    0     ← Different from OR!
```

XOR is like OR but "exclusive" — it excludes the case where both are 1.

Key difference: OR vs XOR
OR: "Is at least one input 1?" — includes the case where both are 1.
XOR: "Is exactly one input 1?" — excludes the case where both are 1.
When A=1 and B=1: OR outputs 1, but XOR outputs 0.

Memory tricks for logic gates

AND gate: Think of it as multiplication.

- 0 × 0 = 0
- 0 × 1 = 0
- 1 × 0 = 0
- 1 × 1 = 1

Anything multiplied by 0 is 0. Only 1 × 1 = 1.

OR gate: Think "any 1 gives 1".

- If you see any 1 in the inputs, output is 1
- Only 0 OR 0 = 0

XOR gate: Think "different = 1".

- Different inputs (0,1 or 1,0) = 1
- Same inputs (0,0 or 1,1) = 0

Real-world examples of logic gates

Gate	Real-world analogy	Example
AND	Both conditions must be true	A car starts only if the key is in AND the clutch is pressed
OR	At least one condition must be true	A burglar alarm sounds if the front door OR back door is opened
NOT	Opposite/inverse	A light is ON when the switch is NOT in the off position
XOR	One or the other, but not both	A two-way light switch — flipping either switch changes the light state

These analogies help understand when each gate would be used in real electronic systems.

Worked Example 1 – Identifying gate outputs

Question: For each logic gate, state the output when A = 1 and B = 0. [4 marks]
(a) AND gate (b) OR gate (c) XOR gate (d) NOT gate with input A

Model answer:

(a) AND: A = 1, B = 0 → Output = 0 (both inputs must be 1) **(1)**
(b) OR: A = 1, B = 0 → Output = 1 (at least one input is 1) **(1)**
(c) XOR: A = 1, B = 0 → Output = 1 (exactly one input is 1) **(1)**
(d) NOT: A = 1 → Output = 0 (inverts the input) **(1)**

Exam tip: Work through each gate's rule systematically. Write down the rule if it helps you remember.

Worked Example 2 – Explaining gate behaviour

Question: Explain the difference between an OR gate and an XOR gate. [3 marks]

Model answer:
An OR gate outputs 1 when at least one input is 1 **(1)**. An XOR gate outputs 1 when exactly one input is 1 **(1)**. They differ when both inputs are 1: OR outputs 1, but XOR outputs 0 **(1)**.

Exam tip: When comparing gates, always mention the specific case where they differ (both inputs = 1 for OR vs XOR).

Worked Example 3 – Completing a truth table

Question: Complete the truth table for an AND gate. [2 marks]

Model answer:

A	B	A AND B
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

(2 marks for all correct; 1 mark for mostly correct with one error)

Exam tip: For AND, remember only the last row (1,1) gives output 1. All other rows give 0.

Worked Example 4 — Identifying a gate from behaviour

Question: A logic gate gives the following outputs. Identify the gate. [1 mark]
When A=0, B=0 → Output=0 | When A=0, B=1 → Output=1 | When A=1, B=0 → Output=1 | When A=1, B=1 → Output=0

Model answer:
XOR gate **(1)**
Reasoning: The output is 1 only when the inputs are different (0,1 or 1,0). When inputs are the same (0,0 or 1,1), the output is 0. This is the behaviour of XOR.

Exam tip: Look at the (1,1) case first — if it outputs 0, it's XOR; if it outputs 1, it could be OR or AND (but AND also needs (0,1) and (1,0) to be 0).

2. Truth tables

A **truth table** shows all possible input combinations and their corresponding outputs for a logic circuit or Boolean expression. For n inputs, there are 2^n possible combinations.

Number of rows in a truth table

Number of inputs	Number of rows	Input combinations
1	$2^1 = 2$	0, 1
2	$2^2 = 4$	00, 01, 10, 11
3	$2^3 = 8$	000, 001, 010, 011, 100, 101, 110, 111

2.1 Constructing truth tables systematically

To construct a truth table correctly, follow these steps:

1. **Count the inputs** and calculate 2^n rows needed
2. **List all input combinations** in binary counting order (000, 001, 010, etc.)
3. **Add columns** for any intermediate results (outputs of individual gates)
4. **Calculate each row** by applying the logic gates in order
5. **Write the final output** in the last column

Systematic input listing for 3 inputs

Always list inputs in binary counting order to ensure you don't miss any combinations:

```
Row | A | B | C |  Binary value
----|---|---|---|--------------
```

```
1 | 0 | 0 | 0 | 0
2 | 0 | 0 | 1 | 1
3 | 0 | 1 | 0 | 2
4 | 0 | 1 | 1 | 3
5 | 1 | 0 | 0 | 4
6 | 1 | 0 | 1 | 5
7 | 1 | 1 | 0 | 6
8 | 1 | 1 | 1 | 7
```

Pattern: Column C alternates every row (0,1,0,1...). Column B alternates every 2 rows (0,0,1,1...). Column A alternates every 4 rows (0,0,0,0,1,1,1,1).

2.2 Truth tables for combined circuits

When a circuit has multiple gates, add intermediate columns to show the output of each gate before calculating the final result.

Example: Truth table for (A AND B) OR C

A	B	C	A AND B	(A AND B) OR C
0	0	0	0	**0**
0	0	1	0	**1**
0	1	0	0	**0**
0	1	1	0	**1**
1	0	0	0	**0**
1	0	1	0	**1**
1	1	0	1	**1**
1	1	1	1	**1**

The intermediate column (A AND B) makes it easier to calculate the final output.

Worked Example 5 — Truth table for combined gates

Question: Complete the truth table for the expression: (NOT A) AND B [4 marks]

Model answer:

A	B	NOT A	(NOT A) AND B
0	0	1	**0**
0	1	1	**1**

A	B	NOT A	(NOT A) AND B
1	0	0	**0**
1	1	0	**0**

(1 mark for correct NOT A column, 1 mark for each pair of correct final outputs)

Exam tip: Always include intermediate columns. They help you avoid errors and can earn method marks even if the final answer is wrong.

Worked Example 6 — Three-input truth table

Question: Complete the truth table for the expression: A OR (B AND C) [4 marks]

Model answer:

A	B	C	B AND C	A OR (B AND C)
0	0	0	0	**0**
0	0	1	0	**0**
0	1	0	0	**0**
0	1	1	1	**1**
1	0	0	0	**1**
1	0	1	0	**1**
1	1	0	0	**1**
1	1	1	1	**1**

Exam tip: For 3-input tables, always use 8 rows. List inputs in binary counting order (000 to 111) to avoid missing combinations.

Worked Example 7 — Interpreting a truth table

Question: A logic circuit has the following truth table. Describe when the output Q is 1. [2 marks]

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

Model answer:
The output Q is 1 only when both inputs A and B are 0 **(1)**. This could be expressed as Q = (NOT A) AND (NOT B) / the output is 1 when A is 0 AND B is 0 **(1)**.

Exam tip: When interpreting, look for patterns. Ask: "When is the output 1?" or "When is the output 0?" and describe the input conditions using the gates you know (NOT, AND, OR, XOR).

Worked Example 8 – Truth table with XOR

Question: Complete the truth table for: (A XOR B) AND C [4 marks]

Model answer:

A	B	C	A XOR B	(A XOR B) AND C
0	0	0	0	**0**
0	0	1	0	**0**
0	1	0	1	**0**
0	1	1	1	**1**
1	0	0	1	**0**
1	0	1	1	**1**
1	1	0	0	**0**
1	1	1	0	**0**

Exam tip: Remember XOR gives 1 when inputs are different. Check: rows 3,4,5,6 have different A and B values, so A XOR B = 1 for those rows.

3. Logic circuits and Boolean expressions

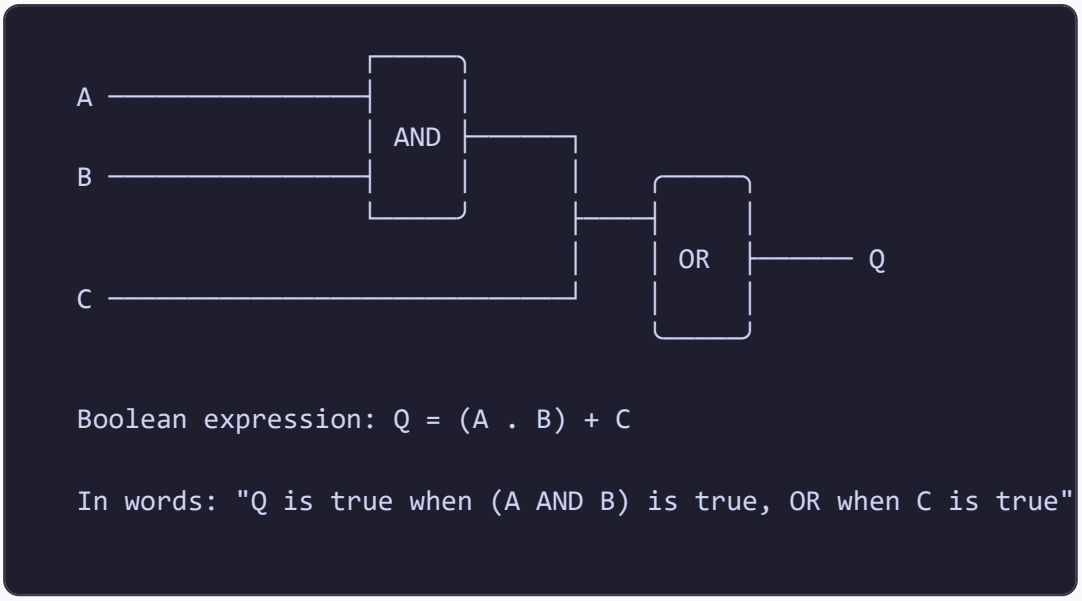
Logic circuits are diagrams showing how logic gates are connected. Boolean expressions are mathematical representations of the same logic using symbols. You need to be able to convert between them.

3.1 Reading logic circuits

To read a logic circuit:

1. **Identify the inputs** (usually on the left, labelled A, B, C, etc.)
2. **Follow each signal path** through the gates from left to right
3. **Note intermediate outputs** where one gate feeds into another
4. **Identify the final output** (usually on the right, often labelled Q)

Example circuit: (A AND B) OR C



3.2 Boolean expression notation

AQA uses specific notation for Boolean expressions:

- **A . B** means A AND B (dot represents AND)
- **A + B** means A OR B (plus represents OR)
- **A ⊕ B** means A XOR B (circled plus represents XOR)
- $\overline{A}$ means NOT A (overbar represents NOT)

> **Order of operations (precedence):**
> 1. NOT (evaluated first)
> 2. AND
> 3. OR (evaluated last)
>
> Use brackets to make the order clear and avoid ambiguity.

3.3 Converting circuit to expression

To convert a logic circuit to a Boolean expression:

1. Start from the inputs and work towards the output

2. Write the expression for each gate's output
3. Combine expressions, using brackets for clarity

Example: Converting circuit to expression

```
A ──────────┌─────┐──○──────┐
            │ NOT │         │
            └─────┘         ├────┌─────┐
                            │    │     │
                            │    │ AND ├──────── Q
B ──────────────────────────┘    │     │
                                 └─────┘

Step 1: NOT gate output = NOT A = Ā
Step 2: AND gate inputs are Ā and B
Step 3: Final expression: Q = Ā . B  (or "NOT A AND B")
```

3.4 Converting expression to circuit

To convert a Boolean expression to a logic circuit:

1. Identify the main operation (outermost, evaluated last)
2. Draw that gate with its output as Q
3. Work backwards to add gates for sub-expressions
4. Connect the inputs

Example: Converting expression to circuit

Expression: $Q = (A + B) . \overline{C}$

```
Analysis:
• Main operation: AND (the dot)
• Left input to AND: (A + B) which is A OR B
• Right input to AND: C̄ which is NOT C

Circuit:

A ──────────────┌────┐
                │    │
                │ OR ├──────┐
B ──────────────└────┘      │
                            ├────┌─────┐
                            │    │     │
                            │    │ AND ├──────── Q
                            │    │     │
C ──────────┌─────┐──○──────┘    └─────┘
            │ NOT │
```

3.5 Common exam question types

Boolean logic questions in the AQA GCSE exam typically fall into these categories:

Question type	What you need to do	Typical marks
Complete a truth table	Fill in missing output values for given inputs	2—4 marks
Construct a truth table	Create entire table from expression or circuit	3—6 marks
Draw a logic circuit	Convert Boolean expression to circuit diagram	2—4 marks
Write Boolean expression	Convert circuit diagram to expression	2—3 marks
Identify a gate	Name gate from its truth table or behaviour	1—2 marks
Explain gate behaviour	Describe when output is 1 or compare gates	2—4 marks

Step-by-step approach for complex circuits

When faced with a multi-gate circuit, follow this systematic approach:

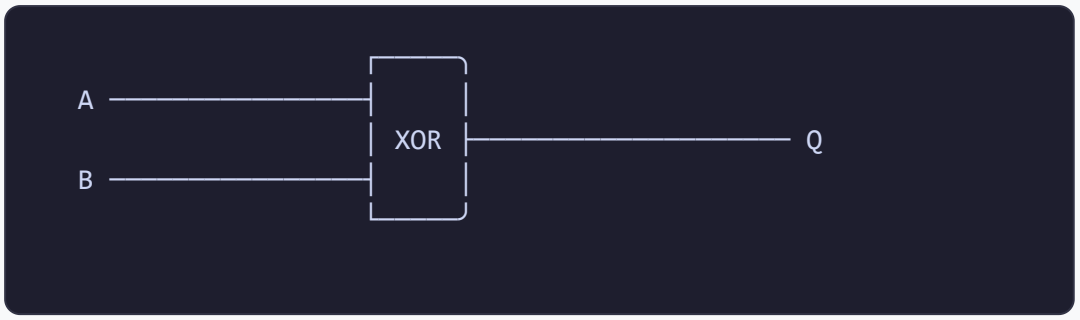
1. **Label intermediate wires:** Give temporary names (P, Q, R) to outputs of inner gates
2. **Work left to right:** Calculate outputs of leftmost gates first
3. **Build up gradually:** Use intermediate results to calculate the next gate
4. **Check your work:** Verify a couple of rows by tracing through the circuit manually

```
Example: Label intermediate outputs

                    ┌─────┐
A ──────────────────┤     │
                    │ AND ├── P ──┐
B ──────────────────┤     │       │
                    └─────┘       ├────┐
                                  │    │ OR ├── Q (final)
C ────────────────────────────────┘    │
                                       └────┘

Step 1: P = A . B (AND gate output)
```

```
Step 2: Q = P + C (OR gate with P and C)
Therefore: Q = (A . B) + C
```

Worked Example 9 — Circuit to expression

Question: Write the Boolean expression for this circuit. [2 marks]



Model answer:

Q = A ⊕ B **(2 marks)**

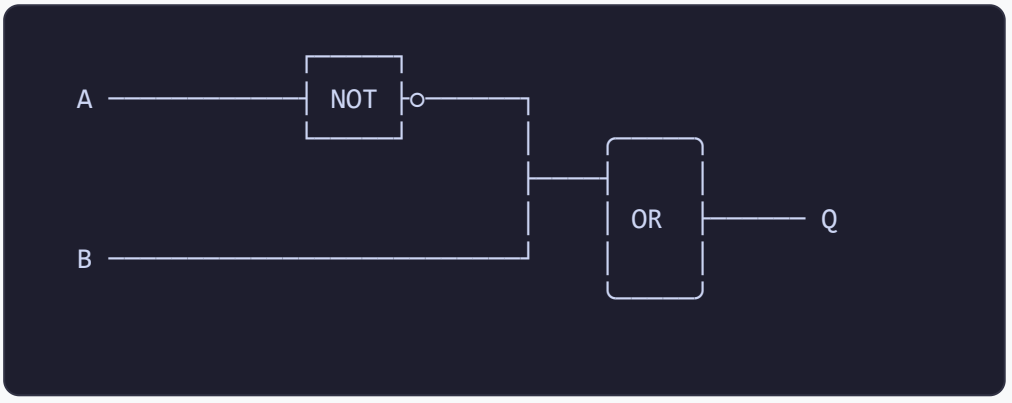
Or written as: Q = A XOR B

Exam tip: Use the correct AQA notation: ⊕ for XOR, . for AND, + for OR, overbar for NOT.

Worked Example 10 — Expression to circuit

Question: Draw a logic circuit for the expression: Q = $\overline{A}$ + B [3 marks]

Model answer:

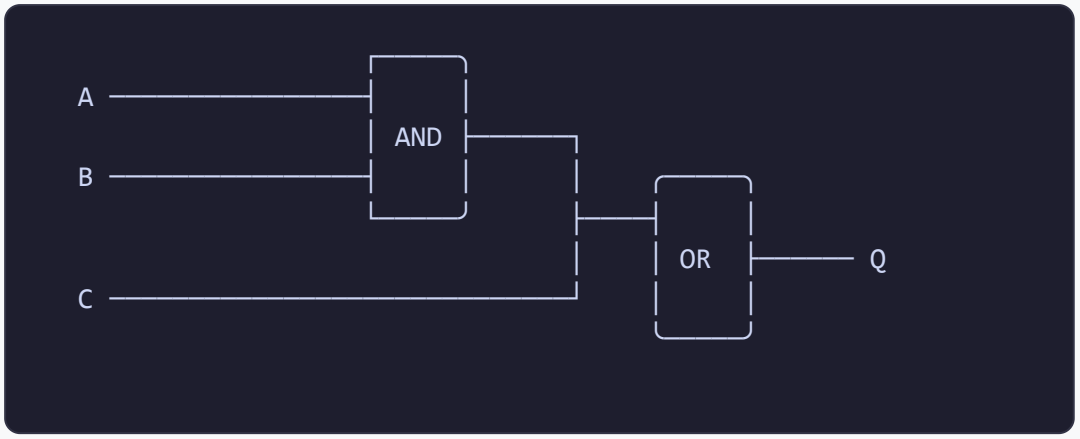


(1 mark for NOT gate on A, 1 mark for OR gate, 1 mark for correct connections)

Exam tip: Draw gates clearly with the correct symbols. Label inputs and output. Use straight lines for connections.

Worked Example 11 — Complex circuit to expression

Question: Write the Boolean expression for this circuit. [3 marks]



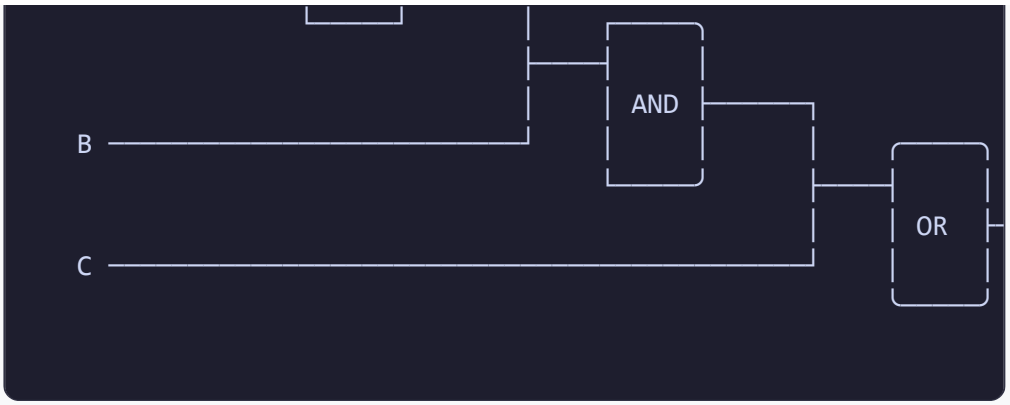
Model answer:
The AND gate takes inputs A and B, producing A . B **(1)**
The OR gate takes (A . B) and C as inputs **(1)**
Final expression: Q = (A . B) + C **(1)**

Exam tip: Work through the circuit gate by gate. Write intermediate expressions as you go.

Worked Example 12 — Complete analysis

Question: For the Boolean expression Q = ($\overline{A}$. B) + C:
(a) Draw the logic circuit. [3 marks]
(b) Complete the truth table. [3 marks]

Model answer (a):



Model answer (b):

A	B	C	$\overline{A}$	$\overline{A}$. B	Q
0	0	0	1	0	**0**
0	0	1	1	0	**1**
0	1	0	1	1	**1**
0	1	1	1	1	**1**
1	0	0	0	0	**0**
1	0	1	0	0	**1**
1	1	0	0	0	**0**
1	1	1	0	0	**1**

Exam tip: In multi-part questions, use your circuit from (a) to help with (b). Work through systematically.

Worked examples summary

The following table summarises the 12 worked examples covered in this topic, organised by section and exam skill.

Example	Focus	Marks	Key skill
1	Identifying gate outputs	4	Apply gate rules to specific inputs
2	Explaining OR vs XOR	3	Compare and contrast gates
3	Completing AND truth table	2	Construct basic truth table
4	Identifying gate from behaviour	1	Recognise gate patterns

Example	Focus	Marks	Key skill
5	Truth table with NOT and AND	4	Combined gates, intermediate columns
6	Three-input truth table	4	8-row table construction
7	Interpreting a truth table	2	Describe behaviour in words
8	Truth table with XOR	4	XOR in combined circuit
9	Simple circuit to expression	2	Read circuit, write expression
10	Expression to circuit	3	Draw circuit from expression
11	Complex circuit to expression	3	Multi-gate circuit analysis
12	Complete analysis	6	Circuit + truth table combined

Quick reference

Logic gate summary

Gate	Inputs	Rule	Expression	Output = 1 when...
NOT	1	Inverts input	$\overline{A}$	Input is 0
AND	2+	All inputs must be 1	A . B	Both A AND B are 1
OR	2+	At least one input is 1	A + B	A OR B (or both) is 1
XOR	2	Exactly one input is 1	$A \oplus B$	Inputs are different

Truth tables (complete)

NOT gate

A	$\overline{A}$
0	**1**
1	**0**

AND gate

A	B	A . B
0	0	**0**
0	1	**0**

A	B	A . B
1	0	**0**
1	1	**1**

OR gate

A	B	A + B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

XOR gate

A	B	A ⊕ B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

Boolean notation

Operation	Symbol	Example	Read as
AND	. (dot)	A . B	"A and B"
OR	+ (plus)	A + B	"A or B"
XOR	⊕ (circled plus)	A ⊕ B	"A exclusive or B"
NOT	overbar	$\overline{A}$	"not A"

Typical pitfalls

Pitfall 1 — Confusing OR and XOR.
Wrong: Thinking OR outputs 0 when both inputs are 1.
Right: OR outputs 1 when at least one input is 1 (including when both are 1). XOR outputs 0 when both are 1.

Pitfall 2 — Missing rows in truth tables.

Wrong: Using 6 rows for a 3-input table.
Right: 3 inputs = 2^3 = 8 rows. Use binary counting to list all combinations.

Pitfall 3 — Forgetting intermediate columns.
Wrong: Trying to calculate complex expressions in one step.
Right: Add columns for intermediate gate outputs. This reduces errors and earns method marks.

Pitfall 4 — Wrong symbol for operations.
Wrong: Using × for AND or v for OR.
Right: AQA uses: . for AND, + for OR, ⊕ for XOR, overbar for NOT.

Pitfall 5 — Confusing + with addition.
Wrong: Thinking A + B means arithmetic addition.
Right: In Boolean logic, + means OR. It's a logical operation, not arithmetic.

Pitfall 6 — Drawing incorrect gate symbols.
Wrong: Using rectangles for all gates.
Right: Learn the standard symbols: D-shape for AND, curved shield for OR, extra curve for XOR, triangle with bubble for NOT.

Pitfall 7 — Not applying NOT first.
Wrong: Evaluating $A . \overline{B}$ as "A AND B, then NOT".
Right: NOT applies only to what's under the bar. $\overline{B}$ means "NOT B", then AND with A.

Pitfall 8 — Random input ordering.
Wrong: Listing inputs in random order (11, 00, 10, 01).
Right: Use binary counting order: 00, 01, 10, 11. This ensures no combinations are missed.

Pitfall 9 — Forgetting brackets in expressions.
Wrong: Writing A . B + C without clarity.
Right: Use brackets to show order: (A . B) + C or A . (B + C). This avoids ambiguity.

Pitfall 10 — Using NAND or NOR.
Wrong: Including NAND or NOR gates in answers.
Right: AQA only requires NOT, AND, OR, and XOR. NAND and NOR are not in the specification.

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.